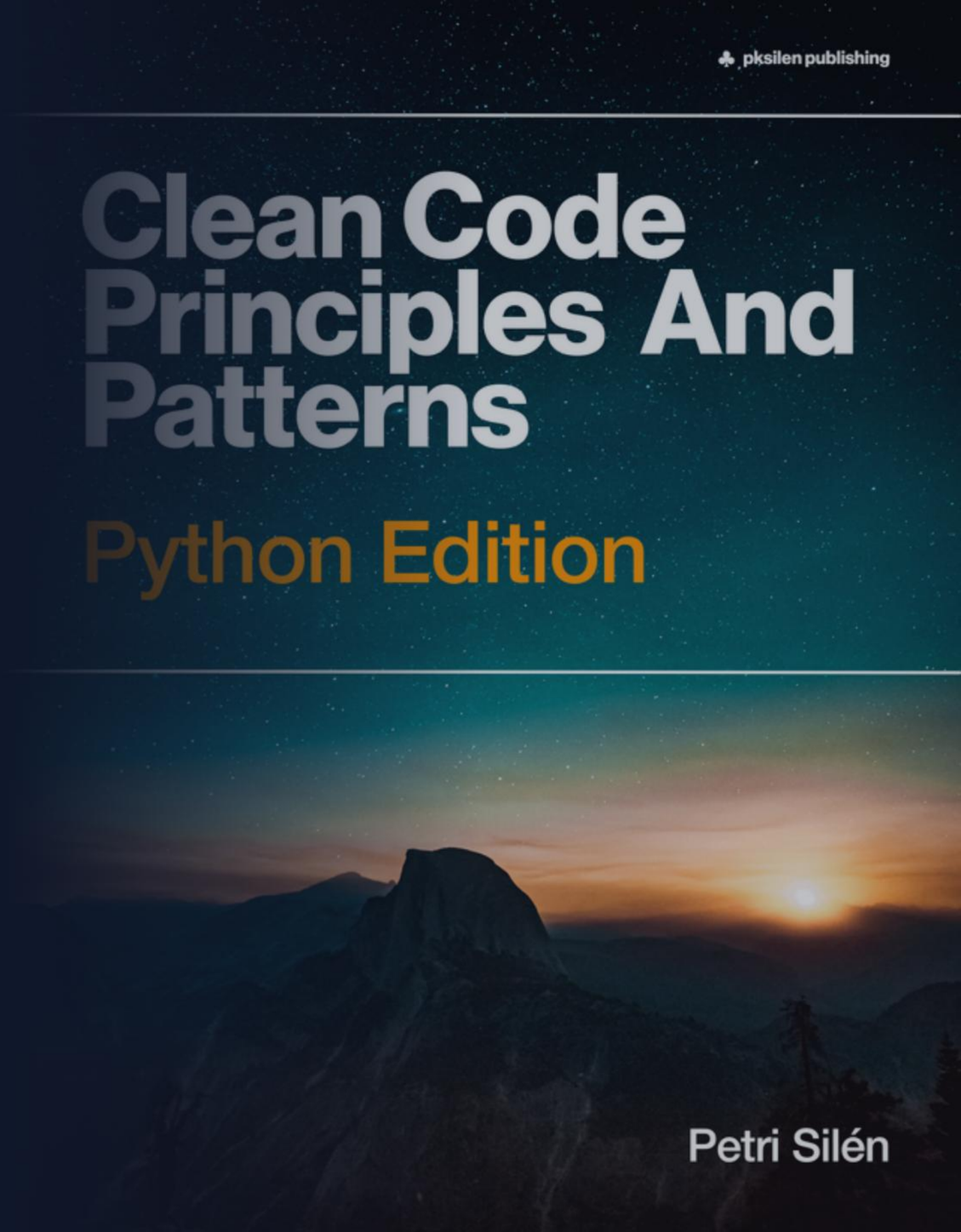
FUNDAMENTOS PYTHON

# Listas, Dicionários e o Laço For

Domine o armazenamento de dados e a iteração automática.



# Parte 1: Listas

## Uma coleção ordenada e mutável de múltiplos elementos.

- Agrupados por colchetes [ ].
- Mantém a ordem de inserção.
- Permite alterações após criação.



# A Estrutura de Índices

O Python começa a contar as posições sempre a partir do ZERO.

| 0 | 1 | 2 |
| --- | --- | --- |
| "maçã" | "uva" | "kiwi" |

# Criando e Acessando

```
# 1. Criando a lista
frutas = ["maçã", "uva",
"kiwi"]

# 2. Acessando informações
print(frutas[0]) # maçã
print(frutas[2]) # kiwi
```

Saída no Terminal:

```
maçã
kiwi
```

# Modificando Elementos

```
# As listas são mutáveis!
# Vamos trocar "maçã" por
"laranja"
frutas[0] = "laranja"
```

**Nova Lista Atualizada:**

```
["laranja", "uva", "kiwi"]
```

# Manipulando a Lista

## .append()

Adiciona um item no final.

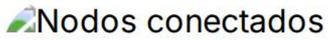
```
frutas.append("limão")
```

## .pop()

Remove e retorna o último item.

```
ultima = frutas.pop()
```

# Parte 2: Dicionários




## Estruturas baseadas em pares de chave e valor.

- 🔑 Busca extremamente rápida por chaves.
- 🔗 Cada chave possui um valor associado.
- Agrupados por chaves **{ }**.

# Criando Dicionários

Mapeie informações usando o formato chave:valor.

```
usuario = {
    "nome": "Carlos",
    "idade": 28,
    "ativo": True
}
```

# Métodos Essenciais de Dicionários

### `.keys()`

Retorna apenas as chaves cadastradas.

### `.values()`

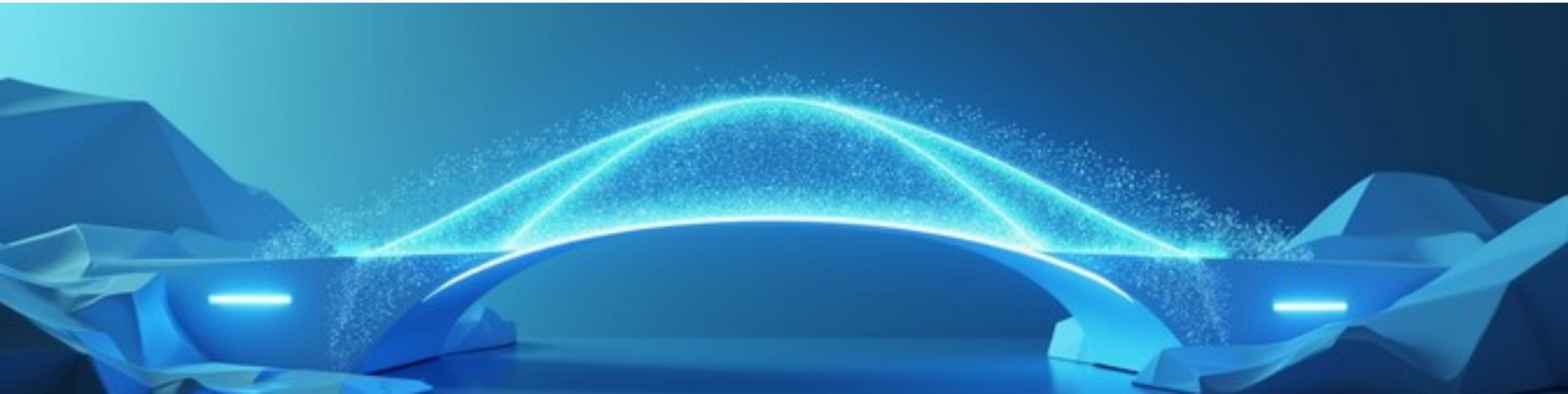
Retorna apenas os valores armazenados.

### `.items()`

Retorna os pares exatos de chave e valor.

# O Problema da Escala

Como processar listas e dicionários com milhares de itens sem ter que digitar um por um?



# Parte 3: O Laço For

## Executa um bloco de código repetidamente para cada item.

- Automatiza tarefas repetitivas.
- Itera perfeitamente sobre coleções.



# A Sintaxe Elegante do Python

```
for item in sequencia:
    # Bloco repetido
    print(item)
```

### Atenção à Indentação

Os espaços no início da linha definem exatamente qual bloco de código pertence ao laço.

# Percorrendo uma Lista

```
frutas = ["maçã", "uva", "kiwi"]

for fruta in frutas:
    print(fruta)
```

```
> Terminal:
maçã
uva
kiwi
```

# Percorrendo um Dicionário

Use o método **`.items()`** para obter chave e valor ao mesmo tempo.

```
usuario = {"nome": "Ana", "cargo": "Dev"}

for chave, valor in usuario.items():
    print(chave, "→", valor)
```

# Repetições Numéricas

A função **`range()`** gera sequências automaticamente.

### `range(5)`

`0, 1, 2, 3, 4`

### `range(2, 6)`

`2, 3, 4, 5`

# Controle de Fluxo

## `break`

Interrompe e encerra o laço imediatamente.

## `continue`

Pula a iteração atual e vai para o próximo item.





# Hora de Praticar

A repetição e as estruturas de dados são o motor da automação em Python.

# Image Sources




https://d2sofvawe08yqg.cloudfront.net/cleancodeprinciplesandpatternspythonedition/s_hero2x?1713287422&1713287422

Source: leanpub.com

---




https://www.lightxeditor.com/blog/wp-content/uploads/2026/08/ChatGPT-Image-Aug-12-2026-07_21_07-PM.webp

Source: www.lightxeditor.com

---



https://graphics.rwth-aachen.de/media/paper_images/None/representative_image.jpg

Source: www.vr.rwth-aachen.de

---



https://images.stockcake.com/public/f/6/d/f6d79963-5682-4290-8118-0d4a73fc7f94_large/luminous-tech-bridge-stockcake.jpg

Source: stockcake.com

---



https://miro.medium.com/v2/resize:fit:2000/1*sao9nlpladkA3dzQig2qHQ.jpeg

Source: medium.com

---



https://img.magnific.com/premium-psd/abstract-3d-graph-with-diverging-paths-upward-trend-soft-studio-lighting-isolated-transparent-background_1281529-65202.jpg?semt=ais_hybrid&w=740&q=80

Source: www.magnific.com